

UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO – UNIVERSIDADE DO ESTADO DO RIO
GRANDE DO NORTE

PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIAS DA COMPUTAÇÃO

Lista de Avaliação

Programação Linear Inteira, Mista e Binária Implementação Computacional via Branch-and-Bound e Branch-and-Cut

Disciplina: Métodos e Modelos de Otimização Linear

Semestre: 2025.1

Docente: _____

Valor total: 10,0 pontos

Discente(s): _____

Data: _____

Instruções Gerais

Esta lista é uma **avaliação prática de implementação computacional**. Não há resolução analítica ou manual: o foco central é o desenvolvimento de **algoritmos funcionais** capazes de modelar e resolver problemas clássicos de Programação Linear Inteira Pura (PLIP), Mista (PLIM) e Binária (PIB) por meio dos métodos **Branch-and-Bound (B&B)** e **Branch-and-Cut (B&C)**, implementados pelos próprios discentes.

1. Organização das Equipes

- A turma será organizada em **equipes de até 3 integrantes**, definidas e comunicadas ao docente até a data indicada no cronograma.
- Cada equipe ficará responsável por **pelo menos um problema** desta lista, sorteado ou escolhido conforme acordado com o docente. Equipes que optarem por resolver mais de um problema poderão obter **pontuação adicional**, a critério do docente.
- Cada integrante da equipe deve ser capaz de explicar e defender individualmente qualquer parte do algoritmo implementado, pois poderão ser feitas **perguntas individuais** durante a apresentação.

2. O que deve ser implementado

- **Entrega obrigatória — Algoritmo (B&B ou B&C):** Cada equipe deve implementar, **do zero**, o algoritmo de Branch-and-Bound ou Branch-and-Cut adequado ao problema sorteado. É permitido o uso de bibliotecas auxiliares de álgebra linear e para resolução de relaxações (ex.: `scipy.optimize.linprog`, `numpy`, PuLP restrito a subproblemas de PL). **Não é permitido** usar solvers de PLI como caixa-preta (CPLEX, Gurobi, OR-Tools, GLPK) na solução principal. O algoritmo deve:
 - a) Receber como entrada os dados do problema (parâmetros, restrições, instância);
 - b) Resolver a relaxação linear em cada nó da árvore de busca;
 - c) Aplicar a estratégia de ramificação escolhida pela equipe (first-fractional, most-infeasible, etc.);
 - d) Realizar poda por infeasibilidade, por limitante e por integralidade;
 - e) Para B&C: gerar e adicionar cortes automaticamente (Gomory, cover cuts, subtour cuts, clique cuts, conforme o problema);
 - f) Retornar a **solução ótima inteira**, o valor da função objetivo, o número de nós explorados e o tempo de execução.
- **Entrega obrigatória — Relatório técnico** (máximo 8 páginas, formato PDF):
 - a) Formulação matemática completa do modelo (variáveis, função objetivo, todas as restrições);
 - b) Descrição da estratégia de B&B ou B&C adotada;
 - c) Exemplos de execução com as instâncias do enunciado;
 - d) Análise de desempenho: nós explorados, tempo e qualidade da solução em pelo menos 3 instâncias de tamanhos distintos.
- **Entrega obrigatória — Repositório Git** (GitHub, GitLab ou similar):
 - Código versionado com histórico de commits significativo;
 - `README.md` com instruções claras de instalação, dependências e execução;
 - Pelo menos **3 casos de teste** documentados e reproduzíveis.

3. Apresentação Oral

- Cada equipe realizará uma **apresentação oral de 15 a 20 minutos**, executando o algoritmo **ao vivo** durante a sessão, com a entrada de dados da instância do enunciado e de pelo menos uma instância adicional escolhida pela equipe.
- A ordem de apresentação será **sorteada** na data da avaliação. Equipes ausentes sem justificativa formal prévia receberão **nota zero**.
- Ao final de cada apresentação, o docente e os demais discentes poderão fazer perguntas à equipe por até **5 minutos**.

4. Desafio Opcional — Interface Gráfica Interativa

Como atividade complementar, valendo até **1,0 ponto extra** na nota final da disciplina, cada equipe poderá desenvolver uma **interface gráfica interativa** para seu algoritmo. A interface deve atender aos seguintes requisitos mínimos:

- **Entrada de dados** pelo usuário via formulário visual (parâmetros, tamanho da instância, restrições);
- **Visualização animada da árvore de B&B ou B&C** em tempo real durante a execução: nós explorados destacados progressivamente, nós podados sinalizados visualmente, e a solução ótima evidenciada ao final;
- **Painel de resultados** exibindo: solução ótima, valor da função objetivo, número de nós explorados, cortes adicionados (para B&C) e tempo de execução;
- A interface pode ser desenvolvida com qualquer tecnologia adequada: **Tkinter**, **PyQt6**, **Streamlit**, aplicação web (Flask + D3.js), **JavaFX**, entre outras.

A entrega do desafio poderá ocorrer em data posterior à apresentação principal, conforme prazo definido pelo docente.

Prazos e Forma de Entrega

- **Definição das equipes e escolha/sorteio de problemas:** até _____.
- **Entrega do repositório Git e relatório técnico (PDF):** até _____, via sistema acadêmico (SIGAA).
- **Apresentações orais com execução ao vivo:** na aula do dia _____, conforme cronograma divulgado pelo docente.
- **Entrega do desafio — interface gráfica (opcional):** até _____.

- Trabalhos entregues com atraso sofrerão desconto de **1,0 ponto por dia** de atraso, até o limite de 5 dias; após este prazo, a entrega não será aceita.

Critérios de Avaliação

Item	Critério	Peso
1	Formulação matemática correta e completa do modelo	20%
2	Corretude, eficiência e organização do algoritmo de B&B ou B&C implementado	35%
3	Qualidade do relatório técnico (clareza, rigor, análise de desempenho)	20%
4	Apresentação oral e domínio individual do conteúdo	15%
5	Organização do repositório, documentação e casos de teste	10%
Total obrigatório		100%
Extra	Interface gráfica interativa (desafio opcional)	—

Dicas de Arquitetura para o Algoritmo

Sugere-se estruturar a implementação nos seguintes módulos independentes:

- 1. Solver de relaxação linear:** resolva cada subproblema de PL por Simplex (pode-se usar `scipy.optimize.linprog` ou implementação própria).
- 2. Gerenciador de nós:** estrutura de dados para a fila de nós ativos — pilha (DFS), fila FIFO (BFS) ou fila de prioridade (best-first search por limitante).
- 3. Estratégia de ramificação:** seleção da variável fracionária a ramificar (first-fractional, most-infeasible, pseudo-cost branching).
- 4. Gerador de cortes** (exclusivo para B&C): implemente ao menos um tipo — cortes de Gomory, cover cuts, subtour cuts ou clique cuts, conforme o problema.
- 5. Módulo de poda:** verifique as três condições — inviabilidade, limitante dominado e integralidade da solução.
- 6. Logger de execução:** registre o histórico completo de nós (limitante, decisão, variável ramificada) para o relatório e para eventual visualização na interface gráfica.

Problema 1 Alocação de Tarefas em Sistema Multiprocessador [PLIP · 1,0 pt]

Tipo: Programação Linear Inteira Pura (PLIP) **Dificuldade:** ★★ Média

Um sistema de computação distribuída possui **3 servidores** (S_1, S_2, S_3) e **4 tarefas** (T_1, T_2, T_3, T_4). Cada tarefa deve ser atribuída a *exatamente* um servidor. A tabela abaixo indica o **tempo de execução** (em segundos) de cada tarefa em cada servidor:

	T_1	T_2	T_3	T_4
S_1	4	5	3	7
S_2	3	6	4	5
S_3	5	4	6	3

O **makespan** do sistema é o tempo máximo de processamento entre os servidores. O objetivo é **minimizar o makespan**, respeitando: (i) cada tarefa é alocada a exatamente um servidor; (ii) a carga de cada servidor não pode exceder **12 segundos**.

- Apresente a **formulação matemática completa** do modelo com variáveis binárias $x_{ij} \in \{0, 1\}$ e variável auxiliar contínua M para o makespan. Inclua todas as restrições (atribuição, capacidade e linearização do máximo).
- Implemente o algoritmo de **Branch-and-Bound** que receba como entrada a matriz de tempos, o número de servidores e a capacidade máxima, e retorne a alocação ótima e o valor do makespan. O algoritmo deve exibir, a cada nó explorado, o limitante inferior atual, o limitante superior (melhor solução inteira encontrada) e a decisão de poda ou ramificação.
- Teste sua implementação com a instância fornecida e com **duas instâncias adicionais** geradas aleatoriamente (mínimo: 5 servidores e 8 tarefas). Reporte o número de nós explorados e o tempo de execução em cada caso.
- No relatório, apresente a **árvore de B&B** gerada para a instância do enunciado, identificando nós podados e a solução ótima encontrada.

Dica de modelagem: Introduza $M = \max_j \sum_i p_{ij} x_{ij}$ e linearize com $\sum_i p_{ij} x_{ij} \leq M$ para cada servidor j . O solver de relaxação pode ser `scipy.optimize.linprog`.

Desafio — Interface Gráfica (opcional, +1,0 pt)

Desenvolva uma interface que permita ao usuário inserir a matriz de tempos, visualize a **árvore de B&B animada** (nós verdes = viáveis, vermelhos = podados, dourado = solução ótima) e exiba ao final um **diagrama de Gantt** com a alocação ótima das tarefas nos servidores.

Problema 2 Seleção de Projetos de Pesquisa com Restrições Lógicas
[PIB · 1,0 pt]

Tipo: Programação Inteira Binária (PIB) **Dificuldade:** ★★ Média

Um comitê de fomento avalia **6 projetos de pesquisa** em computação. Cada projeto i tem custo c_i (R\$ mil) e retorno científico r_i (pontos de impacto). Orçamento total disponível: **R\$ 280 mil**.

Projeto	P_1	P_2	P_3	P_4	P_5	P_6
Custo (R\$ mil)	80	60	90	50	70	100
Impacto (pts)	120	85	140	60	110	160

Restrições lógicas: (i) P_3 só pode ser aprovado se P_1 for aprovado; (ii) P_4 e P_5 são mutuamente exclusivos; (iii) ao menos 2 de $\{P_1, P_2, P_4\}$ devem ser aprovados.

- (a) Apresente a **formulação PIB completa**, com todas as restrições lógicas expressas algebricamente em termos das variáveis $x_i \in \{0, 1\}$.
- (b) Implemente o algoritmo de **Branch-and-Bound** com estratégia *best-first search* (fila de prioridade pelo limitante superior). O algoritmo deve mostrar, em cada nó, os limites superior e inferior e a decisão tomada.
- (c) Implemente também a versão **Branch-and-Cut**, adicionando automaticamente **cortes de Gomory** gerados a partir do quadro Simplex da relaxação linear de cada nó com solução fracionária.
- (d) No relatório, compare o número de nós explorados pelo B&B puro versus o B&C e discuta o impacto dos cortes na eficiência da busca.

Desafio — Interface Gráfica (opcional, +1,0 pt)

Painel com formulário de entrada dos projetos (custo, impacto, restrições editáveis), visualização interativa da **árvore de decisão** (clique em um nó para exibir o subproblema correspondente) e gráfico de barras comparando B&B versus B&C em número de nós explorados.

Problema 3 Seleção de Fluxos em Rede de Datacenter (Knapsack Multidimensional) [PIB · 1,0 pt]

Tipo: Programação Inteira Binária — Knapsack Multidimensional **Dificuldade:** ★★ ★ Alta

Um datacenter seleciona **fluxos de dados** a transmitir por um enlace com duas restrições de capacidade: **banda máxima = 100 Mbps** e **buffer máximo = 25 MB**.

	F_1	F_2	F_3	F_4	F_5	F_6	F_7
Banda (Mbps)	30	20	45	15	35	25	40
Buffer (MB)	8	5	10	3	9	6	7
Prioridade	50	30	70	20	60	40	65

- Apresente a **formulação PIB** (variáveis $x_j \in \{0,1\}$) maximizando a prioridade total sujeita às duas restrições de capacidade.
- Implemente o **Branch-and-Bound** com: regra de ramificação *most-infeasible* (variável com parte fracionária mais próxima de 0,5); limitante superior obtido pela mochila fracionária (greedy por razão prioridade/banda).
- Implemente o **Branch-and-Cut** com geração automática de **cortes de cobertura** (*cover cuts*): dado um conjunto C com $\sum_{j \in C} b_j > B$, adicione $\sum_{j \in C} x_j \leq |C| - 1$.
- Teste com a instância fornecida e com instâncias de **10, 20 e 50 fluxos** (dados aleatórios). Reporte tempo de execução, nós explorados e gap de otimalidade em cada caso.

Dica: A relaxação contínua da mochila é resolvida ordenando itens pela razão prioridade/banda e preenchendo gulodamente até esgotar as capacidades.

Desafio — Interface Gráfica (opcional, +1,0 pt)

Visualizador de rede: represente os fluxos como arestas em um grafo e destaque em tempo real, a cada iteração do B&B, quais fluxos foram selecionados. Exiba barras de progresso das capacidades de banda e buffer.

Problema 4 Escalonamento de Máquinas Virtuais em Nuvem (Bin Packing) [PLIP · 1,0 pt]

Tipo: Programação Linear Inteira Pura — Bin Packing **Dificuldade:** ★★ ★

Alta

Um provedor de nuvem precisa alocar **6 máquinas virtuais** (VMs) em servidores físicos com capacidade de **10 unidades de CPU** cada. Demandas de CPU: $d_1=4$, $d_2=3$, $d_3=5$, $d_4=2$, $d_5=4$, $d_6=3$. Objetivo: **minimizar o número de servidores utilizados**.

Defina $y_j \in \{0, 1\}$ (servidor j utilizado) e $x_{ij} \in \{0, 1\}$ (VM i no servidor j).

- Apresente o **modelo PLI completo**: função objetivo, restrições de capacidade, atribuição única de cada VM e ligação entre x_{ij} e y_j .
- Implemente o **Branch-and-Bound** com: limite inferior inicial $\lceil \sum_i d_i / C \rceil$; heurística de bound superior via *First-Fit Decreasing* (FFD); quebra de simetria (fixar $y_j=0$ se $y_{j-1}=0$).
- Adicione **cortes de cobertura** no B&C para reforçar o modelo e reduza o número de nós explorados. Mostre explicitamente cada corte gerado durante a execução.
- Teste com instâncias de **10, 15 e 20 VMs** com demandas aleatórias. Compare B&B puro, B&C e FFD: nós explorados, tempo e qualidade da solução.

Desafio — Interface Gráfica (opcional, +1,0 pt)

Animação de empacotamento: represente os servidores como “caixas” e as VMs como “blocos coloridos” sendo alocados em tempo real conforme o B&B explora a árvore. Sinalizar visualmente as podas (bloco que desaparece) e a solução ótima final.

Problema 5 Planejamento de Produção com Custos Fixos de Setup [PLIM · 1,0 pt]

Tipo: Programação Linear Inteira Mista (PLIM) **Dificuldade:** ★★ Média

Uma fábrica de semicondutores produz **3 modelos de chips** (A , B , C). Cada modelo incorre em custo fixo de setup apenas se for produzido, além de custo variável por unidade.

Chip	Setup (\$)	C. var. (\$/un.)	Dem. mín. (un.)	Receita (\$/un.)
A	200	8	10	20
B	150	6	15	18
C	300	12	8	30

Restrições: produção total ≤ 60 unidades; horas-máquina ≤ 80 h (A : 1h/un., B : 1,5h/un., C : 2h/un.).

- Apresente a **formulação PLIM** com variáveis contínuas $q_k \geq 0$ e binárias $y_k \in \{0, 1\}$, modelando o setup via Big-M ($q_k \leq M y_k$). Discuta a escolha adequada de M e seu impacto na qualidade da relaxação linear.
- Implemente o **Branch-and-Bound** ramificando *apenas* sobre as variáveis binárias y_k . Em cada nó, resolva o PL resultante (com q_k contínuas) como limitante.
- Implemente o **Branch-and-Cut** adicionando cortes de Gomory automaticamente para variáveis binárias fracionárias.
- No relatório, realize uma **análise de sensibilidade**: varie o custo de setup de C de \$100 a \$400 em passos de \$50 e reporte como o lucro ótimo se altera em cada caso.

Desafio — Interface Gráfica (opcional, +1,0 pt)

Painel de controle de produção com *sliders* para ajustar custos de setup e capacidades em tempo real, com atualização automática da solução ótima e exibição do gráfico de sensibilidade interativo.

Problema 6 Cobertura de Zonas em Rede de Telecomunicações (Set Covering) [PIB · 1,0 pt]

Tipo: Programação Inteira Binária — Set Covering **Dificuldade:** ★★ Média

Uma operadora instala antenas em **5 locais candidatos** para cobrir **8 zonas residenciais** com custo mínimo:

Local	Custo (R\$ mil)	Zonas cobertas
L_1	120	$\{Z_1, Z_2, Z_3, Z_5\}$
L_2	80	$\{Z_2, Z_4, Z_6\}$
L_3	150	$\{Z_1, Z_3, Z_5, Z_7, Z_8\}$
L_4	90	$\{Z_4, Z_6, Z_7\}$
L_5	110	$\{Z_3, Z_5, Z_6, Z_8\}$

- Construa a **matriz de cobertura** $A \in \{0, 1\}^{8 \times 5}$ e apresente o modelo PIB completo.
- Implemente o **Branch-and-Cut**: após resolver a relaxação linear de cada nó, gere automaticamente **cortes de Gomory** para variáveis fracionárias e, quando aplicável, **cortes de clique** identificados no grafo de conflito entre

as zonas.

- (c) O algoritmo deve registrar e exibir cada corte adicionado, informando seu tipo (Gomory ou clique) e o nó em que foi gerado.
- (d) Teste com a instância fornecida e com uma instância de **20 locais candidatos** e **15 zonas** gerada aleatoriamente. Compare o desempenho do B&B puro com o B&C.

Desafio — Interface Gráfica (opcional, +1,0 pt)

Mapa interativo (usando `folium` ou `matplotlib`) com pontos georreferenciados representando zonas e locais candidatos. Ao acionar “Resolver”, as antenas selecionadas são destacadas e os raios de cobertura desenhados animadamente conforme o B&C evolui.

Problema 7 Roteamento de Robô Autônomo em Armazém (TSP) [PLIP · 1,0 pt]

Tipo: Programação Linear Inteira Pura — TSP **Dificuldade:** ★★ ★★ Muito Alta

Um robô autônomo de armazém deve visitar **5 estações de coleta** ($E_1, \dots, E_5$) partindo e retornando à **base** E_0 , minimizando a distância total percorrida. Matriz de distâncias (metros):

	E_0	E_1	E_2	E_3	E_4	E_5
E_0	—	10	15	20	18	25
E_1	10	—	12	18	14	22
E_2	15	12	—	8	16	19
E_3	20	18	8	—	11	14
E_4	18	14	16	11	—	9
E_5	25	22	19	14	9	—

- (a) Apresente a **formulação PLI do TSP** com variáveis $x_{ij} \in \{0, 1\}$, restrições de grau e restrições de eliminação de subcircuitos (formulação MTZ ou DFJ).
- (b) Implemente o **Branch-and-Cut** com: relaxação da formulação MTZ em cada nó; detecção automática de subcircuitos na solução corrente; adição dinâmica de **cortes de subtour** ($\sum_{i \in S, j \notin S} x_{ij} \geq 1$); ramificação sobre a variável x_{ij} mais fracionária.
- (c) Teste com a instância fornecida e com instâncias de **10 e 15 nós** com coordenadas aleatórias em $[0, 100]^2$ (distância euclidiana). Reporte nós

explorados, cortes adicionados e tempo de execução.

- (d) No relatório, compare a solução ótima com a heurística do **vizinho mais próximo**: calcule o gap percentual.

Dica: As restrições MTZ são $u_i - u_j + n x_{ij} \leq n - 1$ para $i \neq j, i, j \neq 0$, com $u_i \in \mathbb{Z}^+$.

Desafio — Interface Gráfica (opcional, +1,0 pt)

Mapa 2D com as estações como pontos. Mostre a rota sendo construída aresta por aresta à medida que o B&C avança. Destaque subcircuitos detectados (vermelho) e os cortes de subtour adicionados (azul tracejado) antes de continuar a busca.

Problema 8 Localização de Servidores de Cache em CDN
[PLIM · 1,0 pt]

Tipo: Programação Linear Inteira Mista — Facility Location **Dificuldade:** ★★ Alta

Uma CDN posiciona **servidores de cache** em **4 cidades candidatas** (C_1, C_2, C_3, C_4) para atender **6 regiões** ($R_1, \dots, R_6$). Custos de atendimento h_{ij} e de instalação f_j (R\$ mil/mês):

Região	C_1	C_2	C_3	C_4
R_1	3	8	6	9
R_2	7	2	5	4
R_3	5	6	2	7
R_4	9	3	8	2
R_5	4	7	3	8
R_6	6	5	7	3
Instalação f_j	30	25	35	20

Restrições: cada região atendida por exatamente um servidor; no máximo **3 cidades** com cache; orçamento $\leq$ R\$ 120 mil.

- (a) Apresente o **modelo PLIM** com $y_j \in \{0, 1\}$ (instalar cache em C_j) e $x_{ij} \in \{0, 1\}$ (região R_i atendida por C_j), com todas as restrições.
- (b) Implemente o **Branch-and-Bound** ramificando sobre as variáveis y_j . Para cada nó, resolva o PL resultante (com x_{ij} relaxadas em $[0, 1]$) como limitante superior.

- (c) Adicione **cortes de Gomory** no B&C para as variáveis y_j fracionárias. Exiba cada corte gerado e o nó correspondente.
- (d) Escale para **10 cidades** e **20 regiões** (dados aleatórios) e reporte o tempo de execução e a qualidade da solução obtida.

Desafio — Interface Gráfica (opcional, +1,0 pt)

Mapa com cidades e regiões dispostas geometricamente. Anime a execução do B&B: mostre quais cidades “ligam” seus servidores de cache e quais regiões são conectadas, com o custo total atualizado em tempo real a cada nó explorado.

Problema 9 Empacotamento de Registros em Blocos de Disco (Cutting Stock) [PLIM · 1,0 pt]

Tipo: Programação Linear Inteira Mista — Cutting Stock **Dificuldade:** ★★ ★ Alta

Um SGBD armazena registros em blocos de disco de tamanho fixo $L = 100$ unidades. Há **5 tipos de registros**:

Tipo	R_1	R_2	R_3	R_4	R_5
Comprimento (un.)	45	30	25	20	15
Demanda mínima	4	6	5	8	10

Objetivo: **minimizar o número total de blocos de disco utilizados.**

- (a) Apresente a **formulação PLIM** com variáveis $z_p \geq 0$ inteiras (número de blocos no padrão p) e as restrições de demanda $\sum_p a_{rp} z_p \geq d_r$ para cada tipo r . Descreva também o **subproblema de geração de colunas** como um problema de mochila.
- (b) Implemente a **geração de colunas** para resolver o Problema Mestre Restrito (PMR): resolva iterativamente o PMR como PL, gere a coluna de custo reduzido negativo via mochila, adicione ao PMR e repita até não haver melhora.
- (c) Implemente o **Branch-and-Price** (B&B com geração de colunas em cada nó) para obter a solução inteira ótima. Descreva claramente como a ramificação é realizada no contexto da geração de colunas.
- (d) No relatório, compare o número de blocos obtido pelo Branch-and-Price versus o arredondamento direto da solução fracionária do PMR final.

Desafio — Interface Gráfica (opcional, +1,0 pt)

Visualização de blocos de disco como retângulos divididos em faixas coloridas por tipo de registro. Anime os padrões sendo gerados iterativamente pela geração de colunas e os blocos sendo preenchidos conforme a solução do Branch-and-Price evolui.

Problema 10 Grade de Horários de Disciplinas (Timetabling) [PLIM · 1,0 pt]

Tipo: Programação Linear Inteira Mista — Timetabling **Dificuldade:** ★★ ★★
Muito Alta

Um departamento de computação aloca **4 disciplinas** (D_1, D_2, D_3, D_4) em **3 salas** (S_1, S_2, S_3) e **4 horários** (H_1, H_2, H_3, H_4):

Disciplina	Sala exigida	Restrição de horário
D_1	S_2 ou S_3	Não pode ser ofertada em H_4
D_2	Qualquer sala	Deve preceder D_4 na grade
D_3	S_3 apenas	Não compartilha horário com D_1
D_4	S_1 ou S_2	Somente em H_3 ou H_4

Pontuações de satisfação w_{ijk} :

	H_1	H_2	H_3	H_4
D_1 em S_2	8	7	9	–
D_2 em S_1	6	8	5	7
D_3 em S_3	9	6	8	7
D_4 em S_2	–	–	10	9

- Apresente a **formulação PLI completa** com $x_{ijk} \in \{0, 1\}$, incluindo restrições de conflito de sala, conflito de horário, precedência e restrições específicas de cada disciplina.
- Implemente o **Branch-and-Cut** com: geração automática do **grafo de conflito** a partir das restrições de sala e horário; extração de cliques maximais para gerar **cortes de clique** ($\sum_{(i,j,k) \in Q} x_{ijk} \leq 1$); cortes de Gomory para variáveis binárias fracionárias; estratégia de seleção de nó *depth-first* com retrocesso.
- Teste com a instância fornecida e com uma instância escalada de **8 disciplinas, 5 salas e 6 horários** (dados aleatórios). Reporte tempo de execução,

nós explorados e valor da solução.

- (d) No relatório, compare os resultados com os obtidos pelo **OR-Tools CP-SAT** (usado apenas como referência de validação) e discuta diferenças de desempenho.

Dica: As restrições de conflito formam um grafo em que dois nós (i, j, k) e (i', j, k) são adjacentes se duas disciplinas distintas disputam a mesma sala e horário. Cliques nesse grafo geram cortes válidos.

Desafio — Interface Gráfica (opcional, +1,0 pt)

Grade de horários interativa: exiba uma tabela sala $\times$ horário e anime o preenchimento das células conforme o B&C atribui disciplinas. Mostre o grafo de conflito ao lado, destacando em vermelho os cortes de clique adicionados a cada iteração. Permita ao usuário editar restrições e re-executar o algoritmo.

Referências e Ferramentas Recomendadas

Referências Bibliográficas

- [1] WOLSEY, L. A. *Integer Programming*, 2nd ed. Wiley, 2020.
- [2] HILLIER, F. S.; LIEBERMAN, G. J. *Introduction to Operations Research*, 11th ed. McGraw-Hill, 2021.
- [3] NEMHAUSER, G. L.; WOLSEY, L. A. *Integer and Combinatorial Optimization*. Wiley-Interscience, 1988.
- [4] ALVES, R.; DELGADO, C. *Programação Linear Inteira*. FEP – Universidade do Porto, 1997.
- [5] WINSTON, W. L. *Operations Research: Applications and Algorithms*, 4th ed. Duxbury Press, 2004.
- [6] APPLEGATE, D. L. ET AL. *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, 2006.

Bibliotecas e Ferramentas para Implementação

Biblioteca / Ferramenta	Linguagem	Uso recomendado
scipy.optimize.linprog	Python	Solver de PL para relaxações internas
numpy	Python	Álgebra linear e manipulação de matrizes
PuLP	Python	Modelagem de PL (apenas relaxações)
networkx	Python	Grafo de conflito, detecção de cliques
matplotlib / plotly	Python	Visualização da árvore B&B e gráficos
streamlit	Python	Interface web interativa (desafio)
tkinter / PyQt6	Python	Interface desktop (desafio)
OR-Tools CP-SAT	Python/C++	Referência para validação dos resultados
GLPK / SCIP	C/Python	B&B de médio porte (referência)
JavaFX / Swing	Java	Interface gráfica alternativa (desafio)

Atenção: OR-Tools, CPLEX e Gurobi podem ser usados **apenas para validar** a solução encontrada pelo algoritmo próprio da equipe. Seu uso como solver principal **não** é permitido e implica **nota zero** na entrega obrigatória.

Estrutura Mínima do Repositório Git

```

repositorio/
  README.md          # Instalação, dependências e execução
  src/
    branch_bound.py  # Algoritmo B&B (ou .java / .cpp)
  
```



```

branch_cut.py      # Algoritmo B&C com geração de cortes
relaxation.py     # Solver de relaxação linear (Simplex)
cuts.py           # Módulo de geração de cortes
gui/              # Interface gráfica (desafio opcional)
  app.py
tests/
  test_instance_1.py # Instância do enunciado
  test_instance_2.py # Instância gerada aleatoriamente
  test_instance_3.py # Instância de estresse (grande porte)
report/
  relatorio.pdf     # Relatório técnico (máx. 8 páginas)

```

“Um algoritmo deve ser visto como uma ferramenta para obter compreensão, não apenas resultados.”

— Donald E. Knuth
